

Cost Evaluation of an AI System

Travel Itinerary Planner · OpenAI gpt-4o-mini · Report generated 19 July 2026

All figures in this report come from sixteen live API transactions recorded in `cost-lab-report.csv`, from a local tiktoken measurement, or from projections that are labelled as such. No figures are estimated or illustrative.

1. The use case

The system is a travel itinerary planner. A user supplies a destination, travel dates, the age of each traveller, any pit stops or transfers, dietary requirements and the kinds of activity they care about. It returns a day-by-day plan that balances activity against rest.

The business relevance is that this is a task nobody wants to do manually and that generic AI does badly. A language model asked for a travel plan reliably over-packs it, and cheerfully schedules a four-year-old and a seventy-year-old for the same nine-hour day. The commercial value is in producing a plan a real group can actually sustain.

2. How the AI is implemented

The model is deliberately not in charge of pacing. A deterministic Python engine first computes an *energy budget* for every day of the trip, derived from the stamina of the group (driven by the youngest and oldest traveller), reductions for arrival and departure days, accumulated fatigue after consecutive active days, and points deducted per hour of transfer travel. Activities carry a fixed intensity cost of 10, 25 or 45 points.

Only then is the model called. It receives each day's budget as a hard limit and fills activity slots within it, returning a schema-validated object rather than prose. The response is checked against the same rules that produced the budgets; a failure triggers one repair call.

This split is a cost decision as much as a design one. Constraining the model reduces invalid responses, and every invalid response costs a second billed call. It also means output quality is measurable rather than a matter of taste, which is what makes the parameter comparison below meaningful.

3. Parameters selected for testing, and why

Temperature (0.0, 0.3, 0.7, 1.0). The API default is 1.0. A planner bound by deterministic rules should not need sampling randomness, so the hypothesis was that lower temperatures would break fewer rules, trigger fewer repair calls, and therefore cost less.

max_completion_tokens (2,000, 4,000, 8,000, uncapped). Selected as a spend circuit-breaker. This parameter carries a specific risk with structured output: if a response is cut off mid-JSON it cannot be parsed, and every generated token is still billed. The test was intended to find where that threshold sits.

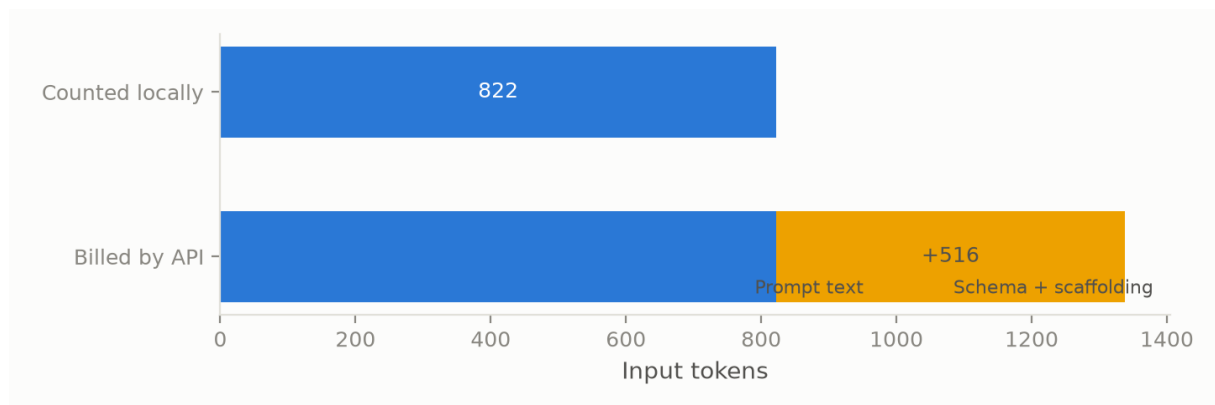
The two were crossed into a 16-cell grid, run against a single fixed trip (Virginia, five days, three travellers aged 4, 35 and 35) so that the parameters were the only variables. One sample per cell.

4. Method: counting tokens and calculating cost

Token counts were captured two independent ways, because they answer different questions.

Locally, with tiktoken. Encoding the actual prompt with the model's BPE vocabulary gives 822 input tokens. This is available before any call is made, so it is what you use to predict cost or to guard against an oversized prompt.

From the API's usage field. The same request was billed at 1,338 input tokens — 516 more, an increase of 63%.



The gap is not an error in either number. It is the JSON schema and message scaffolding that structured output adds server-side, which never appear in the strings encoded locally. The practical lesson is that a local tokenizer materially *understates* a structured-output request: it misses 39% of the input tokens actually billed. A cost forecast built on tokenizer counts alone would be wrong by that margin.

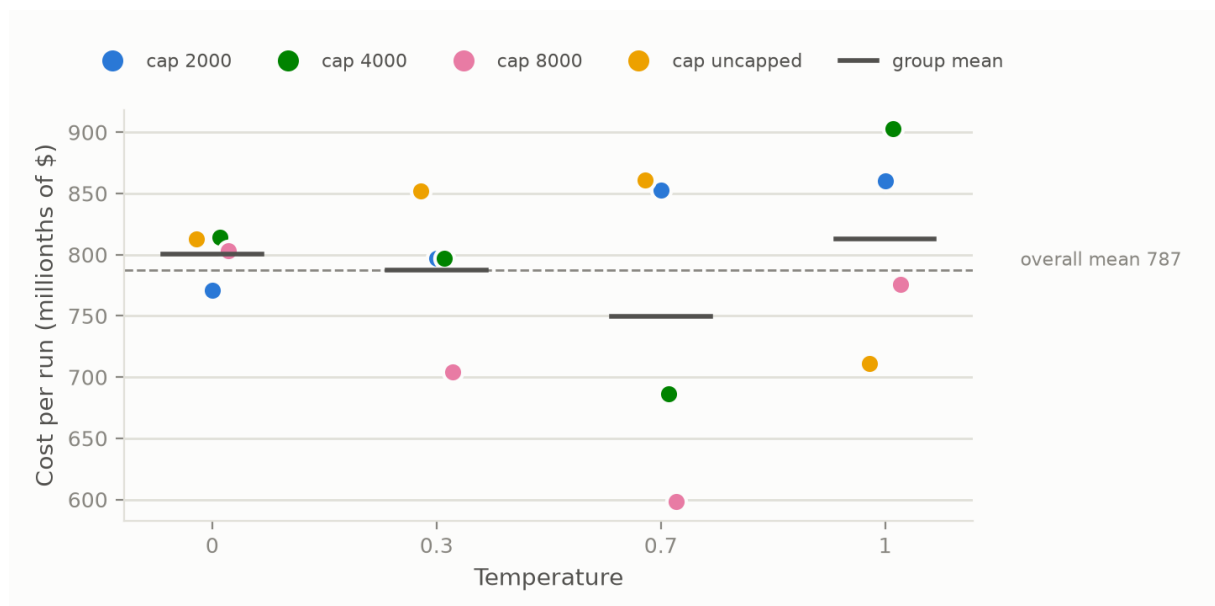
Cost was calculated from published per-million-token rates for the model, applied separately to uncached input, cached input and output tokens.

5. Results of the sixteen transactions

Temp	Cap	Status	Input	Output	Total	Cost (USD)	Viol.	Secs
0	2000	ok	1,338	1,078	2,416	0.000771	0	10.2
0	4000	ok	1,338	1,182	2,520	0.000814	0	10.9
0	8000	ok	1,338	1,164	2,502	0.000803	0	9.7
0	uncapped	ok	1,338	1,180	2,518	0.000813	0	11.6
0.3	2000	ok	1,338	1,154	2,492	0.000797	0	9.5
0.3	4000	ok	1,338	1,153	2,491	0.000797	0	10.3
0.3	8000	ok	1,338	998	2,336	0.000704	0	8.8
0.3	uncapped	ok	1,338	1,245	2,583	0.000852	0	10.4
0.7	2000	ok	1,338	1,248	2,586	0.000853	0	11.5
0.7	4000	ok	1,338	969	2,307	0.000686	0	8.9
0.7	8000	ok	1,338	822	2,160	0.000598	0	7.5
0.7	uncapped	ok	1,338	1,261	2,599	0.000861	0	10.8
1	2000	ok	1,338	1,259	2,597	0.000860	0	11.3
1	4000	ok	1,338	1,331	2,669	0.000903	0	10.9
1	8000	ok	1,338	1,119	2,457	0.000776	0	10.2
1	uncapped	ok	1,338	1,010	2,348	0.000711	0	8.6

Every run succeeded. Zero rule violations, zero truncations and zero repair calls across all sixteen transactions. Total spend was \$0.0126. Input was constant at 1,338 tokens; output ranged 822-1,331, producing costs from \$0.00060 to \$0.00090 — a 1.51x spread.

Temperature	Mean cost	Mean output tokens	Violations	Truncations
0	\$0.00080	1151	0	0
0.3	\$0.00079	1138	0	0
0.7	\$0.00075	1075	0	0
1	\$0.00081	1180	0	0



Each dot is one transaction; the heavy bar is that temperature's mean. The groups overlap almost completely. This is plotted as scattered dots rather than a trend line on purpose — with one sample per cell, joining the means would imply a relationship the data does not support.

6. The headline result is a negative one

Temperature had no measurable effect on either cost or output validity. Mean cost does not move monotonically with temperature, and the cheapest group (0.7) sits between the two coldest settings. The 1.51x spread between the cheapest and priciest run is variation in how long the model chose to make its itinerary, not a consequence of the parameter.

The original hypothesis — that lower temperature would reduce rule violations and therefore repair calls — could not be tested, because *no configuration produced any violations at all*. The most likely explanation is the deterministic budget engine: by the time the model is called, the constraints are narrow enough that sampling randomness has little room to cause a rule breach. Architecture, not parameter tuning, is what removed the repair cost.

The output cap never bound. Peak output across all runs was 1,331 tokens, well under even the 2,000 cap, so no truncation occurred and the parameter was inert for a trip of this length. It would begin to matter on longer trips.

7. An unplanned finding: prompt caching

The recorded costs only reconcile with published rates if a large share of input was billed at the reduced cached rate. Reconstructing the arithmetic, 1,024 input tokens were cached on the first run and 1,280 on every subsequent run — both exact multiples of 128, matching the documented cache granularity. Roughly 96% of input was therefore billed at half price.

This makes the measured costs a best case, not a forecast. The sweep sent one identical prompt sixteen times, which is close to ideal cache behaviour. Real users arrive with different destinations and dates, so a production cache hit rate would be far lower and real cost per itinerary correspondingly higher. The scale model below therefore reports a range rather than a single number.

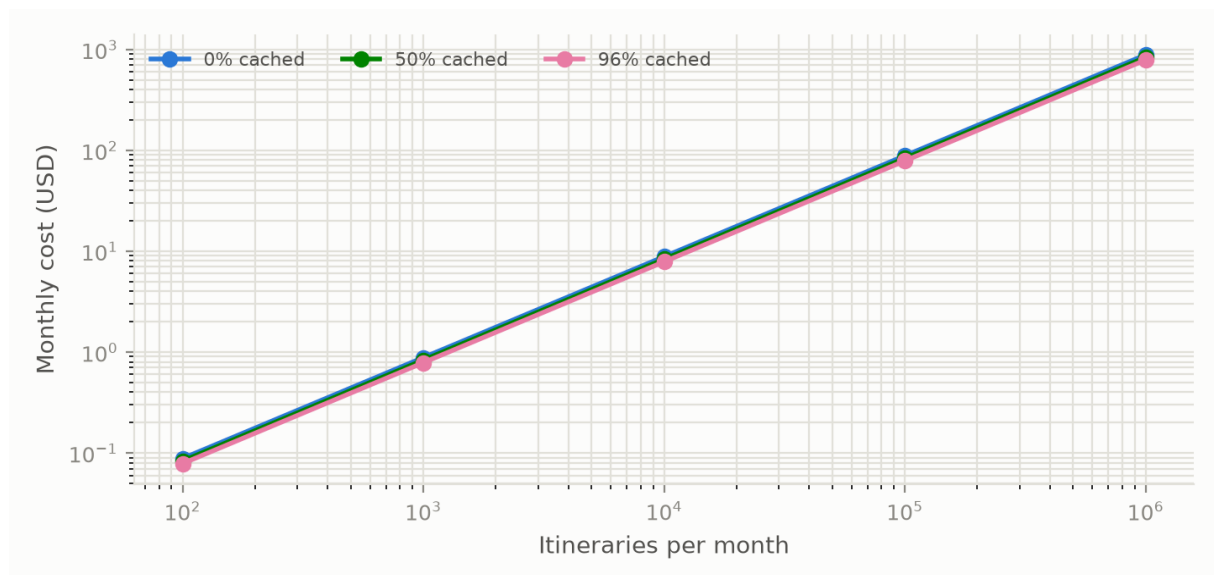
8. Modelling usage at scale

Projections use the measured mean of 1,338 input and 1,136 output tokens per itinerary on gpt-4o-mini. Cache and repair rates are stated assumptions, not measurements — they are the levers worth sensitivity testing.

Scenario	Itineraries/mo	Cache	Repair	Calls/mo	Monthly	Annual
Pilot	100	0%	0%	100	\$0.0882	\$1.0588
Launch	1,000	30%	10%	1,100	\$0.9374	\$11.2490
Growth	10,000	50%	10%	11,000	\$9.1534	\$109.8405
Scale	100,000	70%	10%	110,000	\$89.3260	\$1071.9126

Sensitivity to the cache assumption, at 10,000 itineraries per month:

Cache hit rate	Monthly cost	Annual cost	Cost per itinerary
0% cached	\$8.8230	\$105.8760	\$0.00088
50% cached	\$8.3212	\$99.8550	\$0.00083
96% cached	\$7.8596	\$94.3157	\$0.00079



Both axes are logarithmic. Cost is linear in volume, so the lines are straight; the vertical gap between them is the entire value of prompt caching.

9. How pricing varies across models

The same measured workload (1,338 input + 1,136 output tokens), priced against every model in the project's rate table, uncached:

Model	Cost per itinerary	vs cheapest	At 10,000/month
gpt-4.1-nano	\$0.00059	1.0x	\$5.8820
gpt-4o-mini (used)	\$0.00088	1.5x	\$8.8230
gpt-4.1-mini	\$0.00235	4.0x	\$23.5280
gpt-4.1	\$0.0118	20.0x	\$117.6400
gpt-4o	\$0.0147	25.0x	\$147.0500

The spread between cheapest and dearest is 25x for identical work. At low volume that difference is immaterial; at 100,000 itineraries a month it is the difference between a rounding error and a line item. This is a what-if rather than a measurement: another model would tokenise slightly differently and would produce a different itinerary, possibly of different length.

10. Limitations

One sample per cell. Sixteen runs across sixteen configurations means every cell is a single observation. Differences between configurations of the size seen here are indistinguishable from noise. Three samples per cell would be the minimum for a defensible ranking.

One trip, one model. All runs used a single five-day itinerary on gpt-4o-mini. A fourteen-day trip has a different output profile and might well truncate at caps that were harmless here.

Caching inflates the result favourably. As set out in section 7, the measured per-run cost reflects near-ideal cache reuse that production traffic would not see.

Rates are hardcoded. The pricing table is maintained in source and may be stale; it should be checked against the provider's published pricing before any figure here is relied upon.

11. Summary

How I imagined implementing AI. I built a travel itinerary planner in which the language model is deliberately not in charge. A deterministic Python engine first computes an "energy budget" for each day from traveller ages, arrival and departure days, transfer hours and accumulated fatigue. Only then does the model fill activity slots inside those fixed limits, returning a schema-validated object. That split was a cost decision as much as a design one: an invalid response costs a second billed repair call.

Parameters tested. Temperature (0.0, 0.3, 0.7, 1.0) and max_completion_tokens (2,000, 4,000, 8,000, uncapped) — a 16-run grid over one fixed five-day trip for three travellers, so that only the parameters varied.

Results. All sixteen runs succeeded: zero rule violations, zero truncations, zero repair calls. Input was constant at 1,338 tokens; output ranged 822-1,331, giving costs of \$0.000598-\$0.000903. Mean cost by temperature was \$0.00080, \$0.00079, \$0.00075 and \$0.00081 — no monotonic relationship. The 1.51x spread is output-length noise at one sample per cell, not a parameter effect. Two incidental findings mattered more: prompt caching served roughly 96% of input at half price, and the structured-output schema added 516 tokens (+63%) over a local tiktoken count of the prompt text.

Information needed to decide. Expected monthly volume; a realistic cache hit rate for cold user prompts, since my measurement reused a single prompt and is therefore a best case; the truncation threshold for longer trips; repeated samples to separate signal from noise; and current published pricing, since my rate table is hardcoded and may be stale.

[256 words]